Zarafshon Bank: Data Platform Case Study

Data Engineering & Advanced Analytics · Level 6 · Group 23-302 DA: Abdulhodiy Ataxanov, [member 2], [member 3] · All figures are invented for teaching.

Method: every model, query and load rule below was built and run in PostgreSQL 18; the SQL is in the appendix.

Part 1: Dimensional model for transactions and balances

1.1 Why one fact table cannot serve Questions 1 and 2

Question 1 (daily net inflow) counts **events**: every deposit, withdrawal or transfer is a row, and we add up the amounts. This needs a **transaction fact table** (a transaction grain fact).

Question 2 (average monthly balance) is about a **state**: how much money a customer has at a point in time. A balance is not an event. On a day with no transaction there is no row in the transaction table, but the customer still has a balance. This needs a **periodic snapshot fact table** with one row per account per day.

If we tried to answer Question 2 from the transaction table we would have to replay every transaction since the account was opened. Days without activity would be missing from the average. If we tried to answer Question 1 from the balance table we could only get the change between two days, and we would lose the type, channel and time of each transaction.

1.2 Grain, dimensions and facts

fact_transaction

Grain: one posting leg, meaning one account affected by one transaction.

Dimensions	Facts	Type
dim_date, dim_account, dim_customer, dim_branch, dim_txn_type, dim_channel	amount_uzs (signed)	additive
degenerate: transaction_id, leg_no	fee_uzs	additive
	txn_count (always 1)	additive
	fx_rate_to_uzs	non-additive

fact_daily_balance

Grain: one account per calendar day (end-of-day balance).

Dimensions	Facts	Type
dim_date, dim_account, dim_customer, dim_branch	closing_balance_uzs	**semi-additive** (add across accounts, never across days)
	accrued_interest_uzs	additive
	interest_rate	non-additive

Assumption: all amounts are converted to UZS in the load, and the original currency stays in dim_account.

1.3 Transfers: one row or two?

Decision: two rows (one per leg), with the same `transaction_id` and `leg_no` 1 and 2. The amount is signed: negative for the debited account, positive for the credited account.

Reasons:

- Each leg has its own account, customer, branch and region. One row would need two copies of every dimension key (role-playing `from_` and `to_` keys), and every regional query would need a UNION of both roles.
- Because the grain is "one account affected", `SUM(amount_uzs)` by any dimension is always correct.
- A simple test proves nothing is lost: the legs of one transaction must sum to 0.

Alternative rejected: one row with `from_account_key` and `to_account_key`. It is closer to the source file, but it makes regional sums harder and easy to get wrong.

Example (data in `part1_model.sql`): on 2026-03-02 a customer deposits 50 in Tashkent. On 2026-03-03 a customer sends 100 from Tashkent to Samarkand.

Check	Result
Net inflow, Tashkent city	+50 − 100 = **−50**
Net inflow, Samarkand	**+100**
Net inflow, whole bank	**+50** (the transfer cancels out, only the real cash deposit remains)
Total deposits, Tashkent city (balance table)	3,000 → 2,950, change **−50**
Total deposits, Samarkand (balance table)	500 → 600, change **+100**

The change in total deposits equals the net inflow in every region, so both measures agree.

1.4 Average monthly balance per customer

```
WITH customer_day AS (
    SELECT customer_key, date_key, SUM(closing_balance_uzs) AS balance
    FROM fact_daily_balance
    GROUP BY customer_key, date_key
)
SELECT c.client_no, d.year, d.month_num, ROUND(AVG(cd.balance), 2) AS avg_monthly_balance
FROM customer_day cd
JOIN dim_date d     ON d.date_key = cd.date_key
JOIN dim_customer c ON c.customer_key = cd.customer_key
GROUP BY c.client_no, d.year, d.month_num;
```

The first step adds a customer's accounts for each day (balances are additive across accounts). The second step averages over the days of the month. Result for March 2026: 954.84, 593.55 and 1,883.87.

Why AVG and not SUM: a balance is a *level*, not a *flow*. A customer who holds 1,000 for 31 days does not own 31,000. Summing over days gives 29,600 for the first customer instead of 954.84. That is why the balance is **semi-additive**: it can be summed across accounts, but over time it must be averaged (or taken at the last day).

Warning: averaging the raw table directly would give the average per *account-day*, not per customer-day. That is why the CTE comes first.

1.5 Size of fact_daily_balance and how to keep it affordable

Estimate (assumption: 1.8 million retail + 40,000 business customers, 1.5 accounts each):

- Customers only: 1.84 M × 730 days ≈ **1.3 billion rows**.
- Accounts: 1.84 M × 1.5 = 2.76 M accounts × 730 ≈ **2.0 billion rows**.

Proposal: store a row only when the balance changes (valid from, valid to, balance), instead of one row for every day. The average monthly balance is then a time-weighted average: `SUM(balance × days) / days_in_month`. I tested this on the sample data and it gives exactly the same result as the daily table (954.84, 593.55, 1,883.87). In the sample only 7 of 93 daily rows are real changes (about 8%). If 10–20% of account-days have a change, the table drops to roughly 0.2–0.4 billion rows. Daily rows can still be rebuilt from the intervals with `generate_series` when needed.

Alternatives considered:

- *Keep only month-end balances* (about 66 M rows for 24 months): cheap, but it cannot answer Question 2, because the average needs every day.
- *Keep the full daily table but compress and partition it by month in Parquet*: works and is simple, but it still scans billions of rows. It can be combined with the proposal above.

Part 2: Customer history and slowly changing dimensions

2.1 SCD type for each attribute

Attribute	SCD type	Reason
Branch of record	**Type 2**	Question 5 asks for the full branch history, and branch reports must show a transaction under the branch the customer belonged to at that time.
Segment	**Type 2**	Question 5 asks for it. Segment is used to group inflows, so history must not be rewritten when a customer is upgraded.
Risk rating	**Type 2**	Compliance must know the rating at the time of each transaction (see 2.3), and the Central Bank audits it.
Phone number	**Type 1**	Changes often and nobody analyses old numbers. Overwrite it. The full history of phone numbers stays in the raw vault (Part 4).
Date of first account	**Type 0**	It never changes. If a correction arrives, it fixes an error, so we keep the original loaded value and handle corrections as data-quality issues.
Relationship manager	**Type 1**	It is not needed for Question 5, and staff changes can reassign thousands of customers at once. As Type 2 this would create thousands of new rows for nothing. The history stays in the CRM satellite. If management later wants "sales by manager at the time", use a separate small mini-dimension or a bridge table instead of adding it to dim_customer.

2.2 Rows for customer 4471

Assumption: she is a customer from 2024-01-01 with branch Chilonzor, segment mass, risk low. Branch and segment both changed on 2026-03-12, so they share **one** new version (two changes on the same date do not need two rows).

customer_key	branch	segment	risk_rating	valid_from	valid_to	is_current
4	Chilonzor	mass	low	2024-01-01	2026-03-12	false

customer_key	branch	segment	risk_rating	valid_from	valid_to	is_current
5	Yunusobod	affluent	low	2026-03-12	2026-04-02	false
6	Yunusobod	affluent	medium	2026-04-02	9999-12-31	true

On 2026-04-05, 3 rows exist for her, and exactly 1 row is valid (key 6, which is also the current row). The validity intervals are half-open (`valid_from <= date < valid_to`), so they never overlap and no date belongs to two rows.

2.3 Which transactions were made while she was rated high risk?

```
SELECT f.transaction_id, f.date_key, f.amount_uzs
FROM fact_transaction f
JOIN dim_customer c ON c.customer_key = f.customer_key
WHERE c.client_no = 4471
  AND c.risk_rating = 'high';
```

(In my test data she is never rated high, so I ran the same query with `'medium'`: it returns the two transactions on 2026-04-03 and 2026-04-10.)

Why it is easy: the fact row stores the surrogate key of the version that was valid when it was loaded. That key already points to the right risk rating, so the answer is a plain join with a filter. No date logic is needed.

If the fact had only client_no: every query would need a date-range join:

```
JOIN dim_customer c ON c.client_no = f.client_no
 AND f.txn_date >= c.valid_from AND f.txn_date < c.valid_to
```

This gives the same result (I tested it), but it is slower on billions of rows, because the join cannot use a simple key lookup. It is also easy to get wrong: a forgotten upper bound or overlapping intervals silently duplicates or drops transactions, and every analyst must remember the rule.

2.4 Late-arriving CRM change

Situation: the change effective 2026-03-12 arrives on 2026-03-14. The transactions of 2026-03-12 and 2026-03-13 were already loaded with the old version (mass, Chilonzor).

What the load must do (in one database transaction):

1. Find the version that contains the effective date (2026-03-12).
2. Shorten it: set its `valid_to` to 2026-03-12.
3. Insert the new version from 2026-03-12 until the old `valid_to`, copying every attribute that did not change.
4. Re-key the facts: for that customer, every fact with a date on or after 2026-03-12 that points to the old version is updated to the new key (and branch).
5. Write a line to an audit log (client_no, effective date, rows re-keyed, file name, load time).

Transactions loaded on 2026-03-13 are therefore **corrected** and appear under the affluent segment and Yunusobod branch. Transactions before 2026-03-12 are not touched. I tested this: transaction T-11 (2026-03-13) moved from key 4 to key 5, while T-10 (2026-03-05) stayed on key 4.

Pipeline rule: "A dimension change is applied with its *effective date*, never with its arrival date. If it falls inside an existing version, split that version and re-key the affected facts. The load is idempotent: loading the same file twice changes nothing."

Rejected alternatives:

- *Apply the change from the arrival date (2026-03-14).* The history of 12 to 13 March would be wrong, which is not acceptable for an audited bank.
- *Hold all facts for two days until the CRM file arrives.* It would break the 08:00 deadline for Question 1.

If a fact arrives for a customer that does not exist yet, load it with an inferred member (key -1 or a placeholder row) and fix it when the dimension row arrives, in the same way.

Part 3: Loan lifecycle

3.1 fact_loan_application (accumulating snapshot)

Grain: one loan application, one row for its whole life. The row is created when the application is submitted and updated every time it moves to the next step.

Date keys (one per milestone, all pointing to the same dim_date, so they are role-playing dimensions): `submitted_date_key`, `docs_verified_date_key`, `scored_date_key`, `decision_date_key` (approved or rejected), `signed_date_key`, `disbursed_date_key`, `withdrawn_date_key`. A milestone that has not happened yet points to a special "not yet" row of dim_date (key 99991231). It is never NULL, so joins do not drop rows.

Precomputed lag measures (in days, NULL until both milestones exist): `days_submit_to_docs`, `days_docs_to_scored`, `days_scored_to_decision`, `days_decision_to_signed`, `days_signed_to_disbursed`, `days_approval_to_disbursement`, `days_total_to_disbursement`. These answer "where do applications stall?" with a simple AVG per step (result: mortgages wait longest, 9.0 days to verify documents and 13.0 days from signing to disbursement).

Dimensions: dim_customer (Type 2 key chosen on the submission date), dim_branch, dim_loan_product, dim_employee (the employee responsible now), dim_loan_stage (current stage, with a flag for terminal stages), and the seven role-playing dates. **Degenerate dimension:** `application_no`.

Other facts: requested, approved and disbursed amounts (additive), credit score (non-additive).

Design choice: the age of an open application ("days waiting so far") is **not stored**. It would change every night for every open application and force a rewrite. It is computed at query time from `last_event_ts`.

3.2 Nightly load

1. **Detect changes.** The load keeps a watermark (`last_loaded_at`). It selects only the applications that have at least one source event with `loaded_at` after the watermark. Normally this is a few percent of all rows.
2. **Rebuild only those applications.** For each changed application, all of its events are aggregated again (first date per milestone, last stage, last employee, amounts).
3. **MERGE by application_no.** If the row exists, update: the milestone date keys, current stage, responsible employee, amounts, score, lags and `last_event_ts`. If it does not exist, insert it. The dimension keys (customer, branch, product) are fixed when the application is submitted and are not updated.
4. **Move the watermark** in the same transaction.

How we avoid touching 2 million historical rows: the source events are filtered by the watermark first, so closed applications never enter the join. The fact table can also be partitioned by submission month, so a MERGE on recent months does not scan old partitions. Terminal applications are frozen after a grace period (for example 90 days) and a late correction on them goes through a controlled reload.

Tested result: night 1 loaded 10 applications; night 2 touched only 2 rows (one update, one insert) out of 11; running night 2 again touched 0 rows, so the load is idempotent (safe to re-run).

3.3 Stage counts at the end of every month

The accumulating snapshot keeps only the *current* state and one date per milestone. It can be used to rebuild past stage counts, but badly:

- It needs a long CASE expression over seven date columns for every month end (I wrote it in `part3_queries.sql`).
- It guesses the stage. For example, an application that was approved and then withdrawn is shown as "withdrawn" only from the withdrawal date, and the "rejected vs approved" outcome is taken from the current stage.
- If a date is corrected later, past months silently change, so a published trend chart no longer matches.
- It rescans all applications for every month.

Second table: `fact_loan_stage_month`, a periodic snapshot. Grain: one stage per branch per product per month end. Facts: `application_count` (semi-additive: add across branches and products, never across months), `amount_uzs`. A job inserts the counts on the last day of each month and never changes them afterwards, so the trend chart is stable and fast. (Another option is an event table with one row per stage entry; it can feed both tables, but it is bigger.)

3.4 Median days from approval to disbursement, by branch, Q3 2026

Assumption: "Q3" means the loan was **disbursed** in Q3 (July to September).

```
SELECT b.branch_name,
       COUNT(*) AS loans,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY f.days_approval_to_disbursement) AS median_days
FROM fact_loan_application f
JOIN dim_branch b ON b.branch_key = f.branch_key
JOIN dim_date d   ON d.date_key = f.disbursed_date_key
WHERE d.year = 2026 AND d.month_num BETWEEN 7 AND 9
GROUP BY b.branch_name;
```

Result on the test data: Chilonzor 10 days (3 loans), Registan 11 days (3 loans). A loan disbursed on 5 October is correctly excluded. AVG would be misleading here because a few very slow mortgages pull the average up; the median is stable.

Nightly data-quality test: milestones must be in order. No row may have a later milestone dated before an earlier one, and a loan cannot be disbursed without a signing date:

```
SELECT application_no FROM fact_loan_application
WHERE docs_verified_date_key < submitted_date_key
   OR (scored_date_key < docs_verified_date_key AND scored_date_key <> 99991231)
   OR (decision_date_key < scored_date_key AND decision_date_key <> 99991231)
   OR (signed_date_key < decision_date_key AND signed_date_key <> 99991231)
   OR (disbursed_date_key < signed_date_key AND disbursed_date_key <> 99991231)
   OR (disbursed_date_key <> 99991231 AND signed_date_key = 99991231)
   OR days_approval_to_disbursement < 0;
```

It must return 0 rows. If it returns rows, the pipeline sends the bad applications to a quarantine table, raises an alert for the data owner, and still publishes the rest of the table.

Part 4: Integration layer, Data Vault for four customer sources

4.1 Raw vault design

Business key of `hub_customer`: PINFL. It is the only identifier that all sources can be linked to (through KYC), it never changes, and it is unique per person. `client_no`, `card_holder_id`, `app_user_id` and `crm_id` each exist in one system only, and a person can have several of them over time.

Problem: the card, app and CRM systems do not send PINFL. So the vault has a second hub that keeps the source identifiers exactly as they arrive, and a link that connects them to the person once KYC provides the mapping. This way every source is loaded **raw and immediately**, without waiting for identity matching.

Table	Type	Columns
`hub_customer`	hub	customer_hk (md5 of PINFL), pinfl, load_date, record_source
`hub_source_customer`	hub	src_customer_hk (md5 of system + id), source_system, source_customer_id, load_date, record_source
`hub_account`	hub	account_hk, account_no, load_date, record_source
`link_customer_identity`	link (same-as)	link_hk, customer_hk, src_customer_hk, load_date, record_source (filled from KYC)
`link_customer_account`	link	link_hk, src_customer_hk, account_hk, load_date, record_source
`sat_kyc_customer`	satellite on hub_customer	customer_hk, load_date, hash_diff, record_source, full_name, passport_no, risk_rating, sanctions_flag
`sat_abs_customer`	satellite on hub_source_customer	src_customer_hk, load_date, hash_diff, record_source, branch_code, client_status
`sat_card_customer`	satellite on hub_source_customer	src_customer_hk, load_date, hash_diff, record_source, source_changed_at, address
`sat_app_customer`	satellite on hub_source_customer	src_customer_hk, load_date, hash_diff, record_source, phone, device_id
`sat_crm_customer`	satellite on hub_source_customer	src_customer_hk, load_date, hash_diff, record_source, source_changed_at, segment, address, relationship_manager

Alternative rejected: one single hub with PINFL only, with every source loading straight into it. It would force identity matching **before** the raw load, so a card transaction or app record could not be stored until KYC knows the person. It also breaks the vault principle of loading the data as it arrived.

4.2 Address conflict after 2026-05-03

The card system sends a new address, the CRM still sends the old one. After the load (SQL in `part4_vault.sql`):

`sat_card_customer`

load_date	hash_diff	record_source	address
2025-11-10	ca8f73ec	CARD	Chilonzor 5, Tashkent
2026-05-03	ce413d1a	CARD	Yunusobod 12, Tashkent

`sat_crm_customer` (address column)

load_date	hash_diff	record_source	address
2025-11-11	7a262533	CRM	Chilonzor 5, Tashkent
2026-02-12	23e94bea	CRM	Chilonzor 5, Tashkent (segment changed to affluent)

Nothing was overwritten, because satellites are insert-only. A new row is added only when `hash_diff` (a hash of all descriptive columns) differs from the latest row of the same key. The card system produced a new hash, so a new row was added and the old one stays. The CRM sent no change, so its hash is the same, and a reload on 2026-05-04 inserted 0 rows. The two sources disagree, and the vault stores both disagreements as facts; deciding which one is right is done later, in the business layer.

4.3 Rule for `dim_customer.address`

Rule: the most recent change wins, with a tie-break CRM before CARD. The dimension also gets `address_source` and `address_changed_at`.

```
WITH candidates AS (
    SELECT l.customer_hk, s.address, 'CARD' AS address_source,
           COALESCE(s.source_changed_at, s.load_date) AS changed_at, 2 AS precedence
    FROM sat_card_customer s JOIN link_customer_identity l USING (src_customer_hk)
    UNION ALL
    SELECT l.customer_hk, s.address, 'CRM',
           COALESCE(s.source_changed_at, s.load_date), 1
    FROM sat_crm_customer s JOIN link_customer_identity l USING (src_customer_hk)
), ranked AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_hk ORDER BY changed_at DESC, precedence) AS rn
    FROM candidates WHERE address IS NOT NULL
)
SELECT customer_hk, address, address_source, changed_at AS address_changed_at
FROM ranked WHERE rn = 1;
```

Result for our customer: `Yunusobod 12, Tashkent`, source `CARD`, changed 2026-05-03.

Why this rule: no source is "the truth". CRM addresses are edited by staff and are often stale (as in this example), while the card address comes from the customer. A fixed order "CRM first" would keep showing the old address here. The weakness is that the "most recent" depends on the source change date being correct, so we use the date the source gave us, and fall back to our load date only if it is missing.

4.4 Auditor question

"What did the CRM say this customer's segment was on 2026-02-10, and when did you load it?"

```
SELECT s.segment, s.source_changed_at, s.load_date
FROM sat_crm_customer s
JOIN hub_source_customer h ON h.src_customer_hk = s.src_customer_hk
WHERE h.source_system = 'CRM' AND h.source_customer_id = 'C-2290'
  AND s.source_changed_at <= TIMESTAMP '2026-02-10 23:59:59'
ORDER BY s.source_changed_at DESC
LIMIT 1;
```

Answer: **affluent**, changed in the CRM on 2026-02-08, **loaded by us on 2026-02-12** (the file was late). If the auditor instead asks what *we knew* on 2026-02-10, filter on `load_date` instead, and the answer is **mass**. The vault can answer both questions because it keeps both dates.

Why the star schema cannot: `dim_customer` holds only the cleaned and merged value with a business validity period, so the competing values from losing sources are discarded and we cannot tell what each source said. It also does not record when each value was loaded, and its Type 1 columns and rebuilds overwrite earlier states, so it can never prove what a source file contained at the time.

4.5 A new source: the insurance partner

	Tables to change
(a) Data vault	**Nothing existing changes.** Add one new satellite `sat_ins_customer` and a loader. The partner ID becomes new rows in `hub_source_customer` (source_system = 'INS'), and `link_customer_identity` gets new rows when KYC maps it to a PINFL.
(b) Star only	`dim_customer` must be altered (a new ID column and the new attributes), its load logic and the address/segment precedence rules must be rewritten, and an identity-mapping table has to be added or changed. Past history of the partner cannot be rebuilt if the raw files were not kept.

Conclusion: the vault grows by adding tables, so the new source cannot break existing loads or reports. The star-only design grows by modifying the central dimension that every report depends on. The extra effort of the vault pays off as the number of sources grows.

Part 5: Pipeline, quality and compliance

5.1 Incremental load of the core-banking file

Key used for deduplication: `(transaction_id, leg_no)`. It is the natural grain of `fact_transaction` and has a UNIQUE constraint.

Steps:

1. **Land** the file untouched in a staging table, tagged with `file_id`. Register the file in `load_log`. If the same `file_id` arrives again, the load stops ("skipped: file already loaded"). This covers (a): nothing is loaded twice.
2. **Deduplicate inside the file:** keep only the last row for each key.

3. **Compare with the fact table:** rows that do not exist are inserted; rows that exist with the same amount are ignored; rows with a different amount are corrections.
4. **Corrections (b):** the old amount is first copied to `fact_transaction_audit` (old value, new value, file, time), then the fact row is **updated in place**.
5. **Aggregates (c):** only the days that really changed (new rows or corrections) are recomputed in `agg_daily_inflow` (net inflow by day, region and segment). Yesterday's totals are ready shortly after the load, well before 08:00. A freshness check at 07:00 alerts the team if they are not.

Tested: file F1 inserted 3 rows. File F2 (new day, one correction 500 → 450 for an earlier day, one duplicated line, one re-sent unchanged row) gave `inserted=2 corrected=1 unchanged=1 duplicates_in_file=1`. The aggregate of the earlier day then showed 450, not 500. Loading F2 again changed nothing.

Update, delete-and-insert, or version? I chose **update in place + audit table**.

- *Delete-and-insert* gives the same final numbers, but the fact row gets a new surrogate id, and the old value is lost unless it is logged separately.
- *Versioning the fact rows* (`is_current`) keeps history, but every query on 9 million rows a day must then filter on the flag, and one forgotten filter double-counts the correction. With only 0.5% corrections, a separate small audit table gives the same audit trail without that risk.

5.2 Streaming for Question 3

Pipeline: card processor → **Kafka topic** `card.events` → **stream processor** (for example Flink or Spark Structured Streaming) → **serving table** `rt_card_decline_minute` → dashboard.

- **What is aggregated where:** the stream processor groups events into 1-minute windows by event time and `merchant_category`, and counts `approved` and `declined`. The dashboard reads the small table and sums the minutes to the hour: `declined_share = SUM(declined) / SUM(total)`. Keeping only counts (not ratios) lets any hour or any period be combined correctly, because ratios cannot be added.
- **Event schema:** `event_id` (unique), `event_ts` (event time), `card_token` (token, never the PAN), `merchant_id`, `merchant_category`, `amount`, `currency`, `status` (approved or declined), `response_code`, `decline_reason`, `channel`, `schema_version`.
- **Latency:** a 1-minute window plus a 2-minute watermark gives results within about 3 to 4 minutes, inside the 5-minute target.
- **Duplicates:** the processor drops repeated `event_id` values (state kept for about one hour). The result of each window is written as a full replacement of the row `(window_start, merchant_category)`, so replays do not double-count.
- **Late events:** events inside the watermark are included. Later ones go to a `late` topic. The nightly batch recomputes the whole day from the raw events in Kafka/Parquet and replaces the streaming numbers, so the final figures are exact.

Why not stream the core-banking file: it arrives as one file at 01:00 with end-of-day data, so there are no events to stream, and the answer is only needed by 08:00. It also contains corrections of older days and balances "as of end of day", which suit a batch load with reconciliation. Streaming would add servers, state and failure modes without making any answer arrive earlier.

5.3 Five data-quality tests

#	Category	Test (must return 0 rows)	When it fails
1	Grain	`SELECT transaction_id, leg_no, COUNT(*) FROM fact_transaction GROUP BY 1,2 HAVING COUNT(*) > 1`	**Stop** the load and roll back the batch: duplicates would inflate every total.
2	Referential integrity	`SELECT f.txn_leg_id FROM fact_transaction f LEFT JOIN dim_customer c ON c.customer_key = f.customer_key WHERE c.customer_key IS NULL`	Rows with a missing dimension are **moved to quarantine and mapped to the Unknown member (-1)**; the load continues and an alert is sent to the data owner.
3	No overlapping Type 2 rows	`SELECT a.client_no FROM dim_customer a JOIN dim_customer b ON a.client_no = b.client_no AND a.customer_key <> b.customer_key AND a.valid_from < b.valid_to AND b.valid_from < a.valid_to`	**Stop** the dimension load and keep yesterday's version of the dimension: overlapping rows would double-count facts.
4	Reconciliation	Compare the control record of each file (row count and total amount) with what was loaded: `... WHERE c.row_count <> d.rows_loaded OR c.total_amount <> d.amount_loaded`	**Do not publish** the day. The previous day's aggregates stay visible, a ticket is raised, and the file is requested again from core banking.
5	Freshness	`SELECT MAX(loaded_at) FROM load_log HAVING now() - MAX(loaded_at) > INTERVAL '26 hours'` (run at 07:00)	**Alert** the on-duty engineer; dashboards show a "data is stale" banner so that nobody reads old figures as current.

All five were run on the test data. The first four returned 0 rows. The freshness test returned 0 rows at 08:00 on the day after the load and 1 row two days later, so it detects a missing load.

5.4 Personal data and localisation

Legal check (done on 2026-11-08, from secondary sources only). The Law "On Personal Data" (No. ZRU-547 of 2 July 2019) had, since April 2021, required personal data of Uzbek citizens to be processed on servers physically in Uzbekistan (Article 27-1). Law firms and news sites report that **amendments adopted in March 2026** narrowed this: mandatory local storage now applies to biometric data, genetic data and subscriber data of telecom operators, and other personal data may be processed abroad if conditions are met. A resolution of 29 July 2026 lists 49 countries recognised as adequate for simplified transfers. I could not open the official text on lex.uz, so the **exact wording must be confirmed there** before submission. I also did not verify banking-specific rules (bank secrecy and Central Bank regulations), which may be stricter than the general law.

Design decision: even if the general law no longer forces it for PINFL or phone numbers, Zarafshon Bank keeps **all personal data inside Uzbekistan**. Reasons: Question 6 asks for exactly that proof, KYC may hold biometric data (which is mandatory local), the bank may have stricter sector rules, and one simple rule is easier to prove and audit than category-by-category exceptions.

Data	Where it is stored	Who can query it	How analysts still join customers
PINFL	`hub_customer.pinfl` in the **restricted zone** of the vault (encrypted column)	KYC/compliance officers only, with logged access	Through `customer_hk`, a **keyed hash** (HMAC with a secret key kept in the bank's key vault). Plain MD5 as in my demo is not safe for production, because the 14-digit PINFL space can be brute-forced.
Passport number	`sat_kyc_customer`, restricted zone, encrypted	Compliance only	Not needed for joins; never copied to the star schema.
Full name	`sat_kyc_customer` and a separate `dim_customer_pii` table (restricted)	Compliance and customer service roles	Analysts use `customer_key`; a view with masked name (initials) exists for support.
Phone number	`sat_app_customer` (restricted), masked copy in the star schema	Marketing and support through a masked view (last 4 digits); full number only with approval	`customer_key`, or a tokenised phone value if matching is needed.

Zones: (1) *Restricted raw zone*, which holds identifiers and is encrypted; (2) *business vault and star schema*, which hold only surrogate keys and non-identifying attributes (segment, region, risk rating); (3) *data marts and dashboards*, with no direct identifiers at all. Access is by role, with column masking, row-level rules for branch staff, and an audit log of every query on the restricted zone.

Evidence for the Central Bank (Question 6): (1) the list of servers and storage with their physical address and the data-centre contract/certificate; (2) cloud or hosting configuration showing only Uzbekistan regions, including **backups and disaster recovery**; (3) network diagrams and firewall rules showing no route that sends restricted data abroad, plus egress logs; (4) a statement of where encryption keys live (in the country); (5) access logs proving that administrators work from approved locations; (6) the registration of the databases where the law requires it; (7) an internal audit report repeated every year.

5.5 Cost and operations

- **Daily volume:** 9,000,000 rows × 200 bytes = **1.8 GB per day** before compression.
- **Yearly raw:** 1.8 GB × 365 = **657 GB**.
- **Parquet at 5× compression:** 657 / 5 = **about 131 GB per year**.

Partitioning: partition the fact by transaction date (daily partitions, or monthly for older data), so a daily load and a correction touch one partition and Question 1 reads one partition. Inside each partition sort or cluster by `branch_key`. **Retention (assumption, to be confirmed with the bank's legal team):** keep detailed transactions for 5 years, the last 24 months on fast storage and older months on cheaper storage; keep daily aggregates and balance snapshots longer. At about 131 GB a year, 5 years of detail is under 1 TB compressed, which is small.

Monitoring alert: *row-count anomaly on the daily load.* If the number of rows loaded for yesterday differs by more than 30% from the average of the same weekday in the last four weeks (or the load has not finished by 05:00), page the on-call engineer. It catches an empty, truncated or doubled file before 08:00.

Appendix: SQL

Run in order in one database: part1_model, part1_queries, part2_scd, part3_loans, part3_queries, part4_vault, part4_queries, part5_pipeline, part5_queries.

part1_model.sql

```
-- Part 1: Dimensional model for transactions and balances (Zarafshon Bank)
DROP SCHEMA IF EXISTS bank CASCADE;
CREATE SCHEMA bank;
SET search_path TO bank;

-- ---------- Dimensions ----------
CREATE TABLE dim_date (
    date_key      INTEGER PRIMARY KEY,
    full_date     DATE NOT NULL UNIQUE,
    year          SMALLINT NOT NULL,
    month_num     SMALLINT NOT NULL,
    days_in_month SMALLINT NOT NULL,
    is_month_end  BOOLEAN NOT NULL
);
INSERT INTO dim_date
SELECT TO_CHAR(d,'YYYYMMDD')::INT, d, EXTRACT(YEAR FROM d), EXTRACT(MONTH FROM d),
       EXTRACT(DAY FROM (date_trunc('month', d) + INTERVAL '1 month - 1 day')),
       d = (date_trunc('month', d) + INTERVAL '1 month - 1 day')::DATE
FROM generate_series('2026-01-01'::DATE, '2026-12-31'::DATE, '1 day') d;

CREATE TABLE dim_branch (
    branch_key  SERIAL PRIMARY KEY,
    branch_code TEXT NOT NULL UNIQUE,
    branch_name TEXT NOT NULL,
    region      TEXT NOT NULL
);
INSERT INTO dim_branch (branch_code, branch_name, region) VALUES
('B-001','Chilonzor','Tashkent city'),
('B-002','Yunusobod','Tashkent city'),
('B-010','Registan','Samarkand');

CREATE TABLE dim_customer (
    customer_key SERIAL PRIMARY KEY,
    client_no    BIGINT NOT NULL,              -- natural key (core banking)
    segment      TEXT NOT NULL,                -- mass / affluent / private
    risk_rating  TEXT NOT NULL,
    branch_key   INTEGER NOT NULL REFERENCES dim_branch,
    valid_from   DATE NOT NULL,
    valid_to     DATE NOT NULL DEFAULT '9999-12-31',
    is_current   BOOLEAN NOT NULL DEFAULT TRUE
);
INSERT INTO dim_customer (client_no, segment, risk_rating, branch_key, valid_from) VALUES
(88213, 'mass',     'low', 1, '2020-01-01'),
(91002, 'affluent', 'low', 3, '2020-01-01'),
(93110, 'mass',     'low', 2, '2020-01-01');

CREATE TABLE dim_account (
    account_key  SERIAL PRIMARY KEY,
    account_no   TEXT NOT NULL UNIQUE,
    product_type TEXT NOT NULL,                -- current / savings / term deposit
    currency     CHAR(3) NOT NULL,
    customer_key INTEGER NOT NULL REFERENCES dim_customer,
    branch_key   INTEGER NOT NULL REFERENCES dim_branch
);
INSERT INTO dim_account (account_no, product_type, currency, customer_key, branch_key) VALUES
('A1','current','UZS',1,1),   -- Tashkent, customer 88213
('A2','current','UZS',2,3),   -- Samarkand, customer 91002
('A3','savings','UZS',3,2);   -- Tashkent, customer 93110

CREATE TABLE dim_txn_type (
    txn_type_key SERIAL PRIMARY KEY,
    txn_type     TEXT NOT NULL UNIQUE
);
INSERT INTO dim_txn_type (txn_type) VALUES
('cash deposit'),('cash withdrawal'),('transfer out'),('transfer in'),('card payment'),('fee');

CREATE TABLE dim_channel (
    channel_key SERIAL PRIMARY KEY,
    channel     TEXT NOT NULL UNIQUE
);
INSERT INTO dim_channel (channel) VALUES ('branch'),('atm'),('mobile app'),('card terminal');

-- ---------- Fact 1: transactions ----------
-- Grain: ONE POSTING LEG = one account affected by one transaction.
CREATE TABLE fact_transaction (
    txn_leg_id    BIGSERIAL PRIMARY KEY,
    transaction_id TEXT NOT NULL,              -- degenerate dimension (source id)
    leg_no        SMALLINT NOT NULL,           -- 1 = debited side, 2 = credited side
    date_key      INTEGER NOT NULL REFERENCES dim_date,
    txn_ts        TIMESTAMP NOT NULL,
    account_key   INTEGER NOT NULL REFERENCES dim_account,
    customer_key  INTEGER NOT NULL REFERENCES dim_customer,
    branch_key    INTEGER NOT NULL REFERENCES dim_branch,
    txn_type_key  INTEGER NOT NULL REFERENCES dim_txn_type,
```

```
    channel_key   INTEGER NOT NULL REFERENCES dim_channel,
    amount_uzs    NUMERIC(18,2) NOT NULL,      -- signed: + money in, - money out  (additive)
    fee_uzs       NUMERIC(18,2) NOT NULL DEFAULT 0,  -- (additive)
    txn_count     SMALLINT NOT NULL DEFAULT 1, -- (additive)
    fx_rate_to_uzs NUMERIC(12,4) NOT NULL DEFAULT 1, -- (non-additive)
    UNIQUE (transaction_id, leg_no)
);

-- ---------- Fact 2: daily balances ----------
-- Grain: ONE ACCOUNT PER CALENDAR DAY (end-of-day balance).
CREATE TABLE fact_daily_balance (
    date_key       INTEGER NOT NULL REFERENCES dim_date,
    account_key    INTEGER NOT NULL REFERENCES dim_account,
    customer_key   INTEGER NOT NULL REFERENCES dim_customer,
    branch_key     INTEGER NOT NULL REFERENCES dim_branch,
    closing_balance_uzs NUMERIC(18,2) NOT NULL, -- semi-additive (not over time)
    accrued_interest_uzs NUMERIC(18,2) NOT NULL DEFAULT 0, -- additive
    interest_rate  NUMERIC(6,4) NOT NULL,       -- non-additive
    PRIMARY KEY (date_key, account_key)
);

-- ---------- Sample data ----------
-- 03-02: customer 88213 deposits 50 cash into A1 (Tashkent)
-- 03-03: A1 transfers 100 to A2 (Tashkent -> Samarkand): TWO legs, one transaction_id
-- 03-20: customer 93110 withdraws 300 from A3 at an ATM
INSERT INTO fact_transaction (transaction_id, leg_no, date_key, txn_ts, account_key, customer_key,
                              branch_key, txn_type_key, channel_key, amount_uzs)
SELECT v.tid, v.leg, v.dk, v.ts::TIMESTAMP, a.account_key, a.customer_key, a.branch_key,
       t.txn_type_key, c.channel_key, v.amt
FROM (VALUES
 ('T-1',1,20260302,'2026-03-02 10:15','A1','cash deposit','branch',  50),
 ('T-2',1,20260303,'2026-03-03 11:00','A1','transfer out','mobile app',-100),
 ('T-2',2,20260303,'2026-03-03 11:00','A2','transfer in', 'mobile app', 100),
 ('T-3',1,20260320,'2026-03-20 18:40','A3','cash withdrawal','atm',   -300)
) AS v(tid,leg,dk,ts,acc,typ,ch,amt)
JOIN dim_account a ON a.account_no = v.acc
JOIN dim_txn_type t ON t.txn_type = v.typ
JOIN dim_channel c ON c.channel = v.ch;

-- Daily balances for March 2026 = opening balance + cumulative transactions
INSERT INTO fact_daily_balance (date_key, account_key, customer_key, branch_key, closing_balance_uzs, interest_rate)
SELECT d.date_key, a.account_key, a.customer_key, a.branch_key,
       o.opening + COALESCE((SELECT SUM(t.amount_uzs) FROM fact_transaction t
                             WHERE t.account_key = a.account_key AND t.date_key <= d.date_key), 0),
       0.0500
FROM dim_date d
CROSS JOIN dim_account a
JOIN (VALUES ('A1',1000),('A2',500),('A3',2000)) AS o(account_no, opening) ON o.account_no = a.account_no
WHERE d.year = 2026 AND d.month_num = 3;
```

part1_queries.sql

```
SET search_path TO bank;

\echo '--- 1.3a transfer legs net to zero (test: must return 0 rows)'
SELECT transaction_id, SUM(amount_uzs) FROM fact_transaction
WHERE transaction_id = 'T-2' GROUP BY transaction_id HAVING SUM(amount_uzs) <> 0;

\echo '--- 1.3b net inflow by region, 2026-03-02..03-03'
SELECT b.region, SUM(t.amount_uzs) AS net_inflow
FROM fact_transaction t JOIN dim_branch b ON b.branch_key = t.branch_key
WHERE t.date_key BETWEEN 20260302 AND 20260303
GROUP BY b.region ORDER BY b.region;

\echo '--- 1.3c bank-wide net inflow (transfer must cancel out)'
SELECT SUM(amount_uzs) AS bank_net_inflow FROM fact_transaction WHERE date_key BETWEEN 20260302 AND 20260303;

\echo '--- 1.3d total deposits by region on 03-01 and 03-03, and the reconciliation'
SELECT b.region,
       SUM(f.closing_balance_uzs) FILTER (WHERE f.date_key = 20260301) AS deposits_mar01,
       SUM(f.closing_balance_uzs) FILTER (WHERE f.date_key = 20260303) AS deposits_mar03,
       SUM(f.closing_balance_uzs) FILTER (WHERE f.date_key = 20260303)
     - SUM(f.closing_balance_uzs) FILTER (WHERE f.date_key = 20260301) AS change_in_deposits
FROM fact_daily_balance f JOIN dim_branch b ON b.branch_key = f.branch_key
GROUP BY b.region ORDER BY b.region;

\echo '--- 1.4 average monthly balance per customer (CORRECT: sum accounts per day, then AVG over days)'
WITH customer_day AS (
    SELECT customer_key, date_key, SUM(closing_balance_uzs) AS balance
    FROM fact_daily_balance GROUP BY customer_key, date_key
)
SELECT c.client_no, d.year, d.month_num, ROUND(AVG(cd.balance), 2) AS avg_monthly_balance
FROM customer_day cd
JOIN dim_date d     ON d.date_key = cd.date_key
JOIN dim_customer c ON c.customer_key = cd.customer_key
GROUP BY c.client_no, d.year, d.month_num ORDER BY c.client_no;

\echo '--- 1.4 per branch: average of the daily branch total'
WITH branch_day AS (
    SELECT branch_key, date_key, SUM(closing_balance_uzs) AS balance
    FROM fact_daily_balance GROUP BY branch_key, date_key
)
SELECT br.branch_name, d.year, d.month_num, ROUND(AVG(bd.balance), 2) AS avg_monthly_balance
FROM branch_day bd
```

```
JOIN dim_date d    ON d.date_key = bd.date_key
JOIN dim_branch br ON br.branch_key = bd.branch_key
GROUP BY br.branch_name, d.year, d.month_num ORDER BY br.branch_name;

\echo '--- 1.4 WRONG: SUM over days (31 days counted as 31x the balance)'
SELECT c.client_no, SUM(f.closing_balance_uzs) AS wrong_sum_over_days
FROM fact_daily_balance f JOIN dim_customer c ON c.customer_key = f.customer_key
GROUP BY c.client_no ORDER BY c.client_no;

\echo '--- 1.5 change-only storage: time-weighted average must equal the daily AVG'
WITH changes AS (
    SELECT account_key, date_key, closing_balance_uzs,
           LAG(closing_balance_uzs) OVER (PARTITION BY account_key ORDER BY date_key) AS prev_bal
    FROM fact_daily_balance
), periods AS (
    SELECT account_key, date_key AS from_key, closing_balance_uzs AS balance,
           LEAD(date_key) OVER (PARTITION BY account_key ORDER BY date_key) AS next_from_key
    FROM changes WHERE prev_bal IS DISTINCT FROM closing_balance_uzs
), weighted AS (
    SELECT p.account_key, p.balance,
           (COALESCE(TO_DATE(p.next_from_key::TEXT,'YYYYMMDD'), DATE '2026-04-01')
            - TO_DATE(p.from_key::TEXT,'YYYYMMDD')) AS days
    FROM periods p
)
SELECT a.customer_key, ROUND(SUM(w.balance * w.days) / 31.0, 2) AS time_weighted_avg
FROM weighted w JOIN dim_account a ON a.account_key = w.account_key
GROUP BY a.customer_key ORDER BY a.customer_key;

\echo '--- row counts of change-only vs daily snapshot (sample)'
SELECT (SELECT COUNT(*) FROM fact_daily_balance) AS daily_rows,
       (SELECT COUNT(*) FROM (
           SELECT account_key, closing_balance_uzs,
                  LAG(closing_balance_uzs) OVER (PARTITION BY account_key ORDER BY date_key) AS prev_bal
           FROM fact_daily_balance) x WHERE prev_bal IS DISTINCT FROM closing_balance_uzs) AS change_rows;
```

part2_scd.sql

```
-- Part 2: customer history (SCD Type 2), late-arriving changes, compliance query
SET search_path TO bank;

-- Helper: surrogate key valid on a date (-1 if not found)
CREATE OR REPLACE FUNCTION customer_key_on(p_client_no BIGINT, p_date DATE)
RETURNS INTEGER LANGUAGE sql STABLE AS $$
    SELECT COALESCE((SELECT customer_key FROM dim_customer
                     WHERE client_no = p_client_no
                       AND p_date >= valid_from AND p_date < valid_to), -1);
$$;

-- One current row per client_no
CREATE UNIQUE INDEX IF NOT EXISTS ux_customer_current ON dim_customer (client_no) WHERE is_current;

-- Apply a change effective on a given date. NULL parameter = attribute unchanged.
-- Works for normal AND late-arriving changes: it splits the version that contains
-- the effective date and re-keys the facts that were loaded with the old version.
CREATE OR REPLACE FUNCTION apply_customer_change(
    p_client_no BIGINT, p_effective DATE,
    p_segment TEXT DEFAULT NULL, p_branch_code TEXT DEFAULT NULL, p_risk TEXT DEFAULT NULL)
RETURNS VOID LANGUAGE plpgsql AS $$
DECLARE
    old_row dim_customer%ROWTYPE;
    new_key INTEGER;
    new_branch INTEGER;
BEGIN
    SELECT * INTO old_row FROM dim_customer
     WHERE client_no = p_client_no AND p_effective >= valid_from AND p_effective < valid_to
     FOR UPDATE;
    IF NOT FOUND THEN RAISE EXCEPTION 'no version of % covers %', p_client_no, p_effective; END IF;
    IF old_row.valid_from = p_effective THEN
        RAISE EXCEPTION 'version already starts on % - update it instead', p_effective;
    END IF;

    SELECT COALESCE((SELECT branch_key FROM dim_branch WHERE branch_code = p_branch_code), old_row.branch_key)
      INTO new_branch;

    -- 1) shorten the old version (it keeps its row, just ends earlier)
    UPDATE dim_customer SET valid_to = p_effective, is_current = FALSE
     WHERE customer_key = old_row.customer_key;

    -- 2) insert the new version for [effective, old valid_to)
    INSERT INTO dim_customer (client_no, segment, risk_rating, branch_key, valid_from, valid_to, is_current)
    VALUES (p_client_no, COALESCE(p_segment, old_row.segment), COALESCE(p_risk, old_row.risk_rating),
            new_branch, p_effective, old_row.valid_to, old_row.is_current)
    RETURNING customer_key INTO new_key;

    -- 3) re-key facts that already used the old version on or after the effective date
    UPDATE fact_transaction
       SET customer_key = new_key, branch_key = new_branch
     WHERE customer_key = old_row.customer_key
       AND date_key >= TO_CHAR(p_effective, 'YYYYMMDD')::INT;
END $$;

-- ---------- Scenario for customer 4471 ----------
-- Assumption: she became a customer on 2024-01-01 (Chilonzor, mass, low risk).
INSERT INTO dim_customer (client_no, segment, risk_rating, branch_key, valid_from)
```

```
SELECT 4471, 'mass', 'low', branch_key, DATE '2024-01-01' FROM dim_branch WHERE branch_code = 'B-001';

INSERT INTO dim_account (account_no, product_type, currency, customer_key, branch_key)
SELECT 'A4', 'current', 'UZS', customer_key_on(4471, '2024-01-01'), 1 FROM dim_branch LIMIT 1;

-- Transactions loaded as they arrive, using the version known at load time
INSERT INTO fact_transaction (transaction_id, leg_no, date_key, txn_ts, account_key, customer_key,
                              branch_key, txn_type_key, channel_key, amount_uzs)
SELECT v.tid, 1, v.dk, v.ts::TIMESTAMP, a.account_key, customer_key_on(4471, TO_DATE(v.dk::TEXT,'YYYYMMDD')),
       1, t.txn_type_key, c.channel_key, v.amt
FROM (VALUES ('T-10',20260305,'2026-03-05 09:00',200),
             ('T-11',20260313,'2026-03-13 09:30',100)) AS v(tid,dk,ts,amt)
JOIN dim_account a ON a.account_no = 'A4'
JOIN dim_txn_type t ON t.txn_type = 'cash deposit'
JOIN dim_channel c ON c.channel = 'branch';

\echo '=== before the late CRM file: T-11 (03-13) carries the OLD version'
SELECT f.transaction_id, f.date_key, f.customer_key, c.segment
FROM fact_transaction f JOIN dim_customer c USING (customer_key)
WHERE c.client_no = 4471 ORDER BY f.date_key;

-- 2026-03-14: the CRM file with the 03-12 change (branch + segment) arrives late
BEGIN;
SELECT apply_customer_change(4471, DATE '2026-03-12', 'affluent', 'B-002', NULL);
COMMIT;

\echo '=== after the late file: T-11 is re-keyed, T-10 is untouched'
SELECT f.transaction_id, f.date_key, f.customer_key, c.segment, b.branch_name
FROM fact_transaction f JOIN dim_customer c USING (customer_key) JOIN dim_branch b ON b.branch_key = f.branch_key
WHERE c.client_no = 4471 ORDER BY f.date_key;

-- 2026-04-02: risk rating becomes medium (normal, on-time change)
SELECT apply_customer_change(4471, DATE '2026-04-02', NULL, NULL, 'medium');

-- Later transactions get the newest version at load time
INSERT INTO fact_transaction (transaction_id, leg_no, date_key, txn_ts, account_key, customer_key,
                              branch_key, txn_type_key, channel_key, amount_uzs)
SELECT v.tid, 1, v.dk, v.ts::TIMESTAMP, a.account_key, customer_key_on(4471, TO_DATE(v.dk::TEXT,'YYYYMMDD')),
       2, t.txn_type_key, c.channel_key, v.amt
FROM (VALUES ('T-12',20260403,'2026-04-03 12:00',-50),
             ('T-13',20260410,'2026-04-10 15:20',-75)) AS v(tid,dk,ts,amt)
JOIN dim_account a ON a.account_no = 'A4'
JOIN dim_txn_type t ON t.txn_type = 'card payment'
JOIN dim_channel c ON c.channel = 'card terminal';

\echo '=== 2.2 rows of dim_customer for 4471'
SELECT customer_key, segment, risk_rating, branch_key, valid_from, valid_to, is_current
FROM dim_customer WHERE client_no = 4471 ORDER BY valid_from;

\echo '=== 2.2 rows existing / valid on 2026-04-05'
SELECT COUNT(*) AS rows_total,
       COUNT(*) FILTER (WHERE DATE '2026-04-05' >= valid_from AND DATE '2026-04-05' < valid_to) AS rows_valid_on_that_day
FROM dim_customer WHERE client_no = 4471;

\echo '=== 2.3 transactions made while rated MEDIUM (surrogate key)'
SELECT f.transaction_id, f.date_key, f.amount_uzs
FROM fact_transaction f JOIN dim_customer c ON c.customer_key = f.customer_key
WHERE c.client_no = 4471 AND c.risk_rating = 'medium' ORDER BY f.date_key;

\echo '=== 2.3 same question if the fact had only client_no (date-range join)'
SELECT f.transaction_id, f.date_key, f.amount_uzs
FROM (SELECT t.*, 4471::BIGINT AS client_no FROM fact_transaction t JOIN dim_account a USING (account_key)
       WHERE a.account_no = 'A4') f
JOIN dim_customer c ON c.client_no = f.client_no
 AND TO_DATE(f.date_key::TEXT,'YYYYMMDD') >= c.valid_from
 AND TO_DATE(f.date_key::TEXT,'YYYYMMDD') <  c.valid_to
WHERE c.risk_rating = 'medium' ORDER BY f.date_key;

\echo '=== quality: overlapping versions (must be 0 rows)'
SELECT a.client_no FROM dim_customer a JOIN dim_customer b
  ON a.client_no = b.client_no AND a.customer_key <> b.customer_key
 AND a.valid_from < b.valid_to AND b.valid_from < a.valid_to;
\echo '=== quality: facts pointing to a version not valid on their date (must be 0 rows)'
SELECT f.txn_leg_id FROM fact_transaction f
JOIN dim_date d ON d.date_key = f.date_key
JOIN dim_customer c ON c.customer_key = f.customer_key
WHERE NOT (d.full_date >= c.valid_from AND d.full_date < c.valid_to);
```

part3_loans.sql

```
-- Part 3: Loan lifecycle - accumulating snapshot fact
SET search_path TO bank;
DROP TABLE IF EXISTS fact_loan_application, src_loan_event, etl_watermark, dim_loan_product, dim_loan_stage, dim_employee CASCADE;

-- "Not yet happened" member of the date dimension
INSERT INTO dim_date VALUES (99991231, '9999-12-31', 9999, 12, 31, TRUE) ON CONFLICT DO NOTHING;

CREATE TABLE dim_loan_product (
    loan_product_key SERIAL PRIMARY KEY,
    product_code TEXT NOT NULL UNIQUE,
    product_name TEXT NOT NULL
);
INSERT INTO dim_loan_product (product_code, product_name) VALUES
('CONS','Consumer loan'),('MORT','Mortgage'),('AUTO','Auto loan');
```

```
CREATE TABLE dim_loan_stage (
    stage_key   SERIAL PRIMARY KEY,
    stage_name  TEXT NOT NULL UNIQUE,
    stage_order SMALLINT NOT NULL,
    is_terminal BOOLEAN NOT NULL
);
INSERT INTO dim_loan_stage (stage_name, stage_order, is_terminal) VALUES
('submitted',1,FALSE),('docs_verified',2,FALSE),('scored',3,FALSE),('approved',4,FALSE),
('signed',5,FALSE),('disbursed',6,TRUE),('rejected',7,TRUE),('withdrawn',8,TRUE);

CREATE TABLE dim_employee (
    employee_key SERIAL PRIMARY KEY,
    employee_code TEXT NOT NULL UNIQUE
);
INSERT INTO dim_employee (employee_code) VALUES ('E-01'),('E-02'),('E-03');

-- ---------- Source: one row per step, as core banking records it ----------
CREATE TABLE src_loan_event (
    application_no TEXT NOT NULL,
    stage          TEXT NOT NULL,
    event_ts       TIMESTAMP NOT NULL,
    employee_code  TEXT NOT NULL,
    client_no      BIGINT,            -- filled on 'submitted'
    branch_code    TEXT,              -- filled on 'submitted'
    product_code   TEXT,              -- filled on 'submitted'
    amount_uzs     NUMERIC(18,2),     -- requested / approved / disbursed amount
    credit_score   SMALLINT,          -- filled on 'scored'
    loaded_at      TIMESTAMP NOT NULL,
    PRIMARY KEY (application_no, stage)
);

-- ---------- Accumulating snapshot ----------
-- Grain: ONE LOAN APPLICATION (one row for its whole life, updated as it moves).
CREATE TABLE fact_loan_application (
    application_no          TEXT PRIMARY KEY,                     -- degenerate dimension
    customer_key            INTEGER NOT NULL REFERENCES dim_customer,
    branch_key              INTEGER NOT NULL REFERENCES dim_branch,
    loan_product_key        INTEGER NOT NULL REFERENCES dim_loan_product,
    employee_key            INTEGER NOT NULL REFERENCES dim_employee, -- responsible now
    current_stage_key       INTEGER NOT NULL REFERENCES dim_loan_stage,
    -- one role-playing date key per milestone; 99991231 = not reached yet
    submitted_date_key      INTEGER NOT NULL DEFAULT 99991231 REFERENCES dim_date,
    docs_verified_date_key  INTEGER NOT NULL DEFAULT 99991231 REFERENCES dim_date,
    scored_date_key         INTEGER NOT NULL DEFAULT 99991231 REFERENCES dim_date,
    decision_date_key       INTEGER NOT NULL DEFAULT 99991231 REFERENCES dim_date,  -- approved OR rejected
    signed_date_key         INTEGER NOT NULL DEFAULT 99991231 REFERENCES dim_date,
    disbursed_date_key      INTEGER NOT NULL DEFAULT 99991231 REFERENCES dim_date,
    withdrawn_date_key      INTEGER NOT NULL DEFAULT 99991231 REFERENCES dim_date,
    -- facts
    requested_amount_uzs    NUMERIC(18,2),   -- additive
    approved_amount_uzs     NUMERIC(18,2),   -- additive
    disbursed_amount_uzs    NUMERIC(18,2),   -- additive
    credit_score            SMALLINT,        -- non-additive
    -- precomputed lags in days (NULL until both milestones exist), all additive-as-days
    days_submit_to_docs     INTEGER,
    days_docs_to_scored     INTEGER,
    days_scored_to_decision INTEGER,
    days_decision_to_signed INTEGER,
    days_signed_to_disbursed INTEGER,
    days_approval_to_disbursement INTEGER,
    days_total_to_disbursement INTEGER,
    last_event_ts           TIMESTAMP NOT NULL
);

CREATE TABLE etl_watermark (job TEXT PRIMARY KEY, last_loaded_at TIMESTAMP NOT NULL);
INSERT INTO etl_watermark VALUES ('loan_snapshot', '1900-01-01');

-- ---------- Nightly load: only applications with NEW events are touched ----------
CREATE OR REPLACE FUNCTION load_loan_snapshot(p_run_ts TIMESTAMP)
RETURNS INTEGER LANGUAGE plpgsql AS $$
DECLARE
    v_since TIMESTAMP;
    v_rows  INTEGER;
BEGIN
    SELECT last_loaded_at INTO v_since FROM etl_watermark WHERE job = 'loan_snapshot' FOR UPDATE;

    MERGE INTO fact_loan_application f
    USING (
        WITH changed AS (                    -- applications with at least one new event
            SELECT DISTINCT application_no FROM src_loan_event
             WHERE loaded_at > v_since AND loaded_at <= p_run_ts
        ), a AS (                            -- rebuild the whole life of ONLY those applications
            SELECT e.application_no,
                   MAX(client_no)     FILTER (WHERE stage = 'submitted') AS client_no,
                   MAX(branch_code)   FILTER (WHERE stage = 'submitted') AS branch_code,
                   MAX(product_code)  FILTER (WHERE stage = 'submitted') AS product_code,
                   (MIN(event_ts) FILTER (WHERE stage = 'submitted'))::DATE     AS d_sub,
                   (MIN(event_ts) FILTER (WHERE stage = 'docs_verified'))::DATE AS d_doc,
                   (MIN(event_ts) FILTER (WHERE stage = 'scored'))::DATE        AS d_scr,
                   (MIN(event_ts) FILTER (WHERE stage IN ('approved','rejected')))::DATE AS d_dec,
                   (MIN(event_ts) FILTER (WHERE stage = 'signed'))::DATE        AS d_sig,
                   (MIN(event_ts) FILTER (WHERE stage = 'disbursed'))::DATE     AS d_dis,
                   (MIN(event_ts) FILTER (WHERE stage = 'withdrawn'))::DATE     AS d_wdr,
                   MAX(amount_uzs) FILTER (WHERE stage = 'submitted')  AS req_amt,
                   MAX(amount_uzs) FILTER (WHERE stage = 'approved')   AS apr_amt,
```

```
                  MAX(amount_uzs) FILTER (WHERE stage = 'disbursed')  AS dis_amt,
                  MAX(credit_score) AS score,
                  (ARRAY_AGG(stage ORDER BY event_ts DESC, stage))[1]         AS last_stage,
                  (ARRAY_AGG(employee_code ORDER BY event_ts DESC, stage))[1] AS last_emp,
                  MAX(event_ts) AS last_ts
            FROM src_loan_event e JOIN changed c USING (application_no)
            GROUP BY e.application_no
        )
        SELECT a.*,
               customer_key_on(a.client_no, a.d_sub) AS customer_key,
               b.branch_key, p.loan_product_key, em.employee_key, s.stage_key
        FROM a
        JOIN dim_branch b       ON b.branch_code = a.branch_code
        JOIN dim_loan_product p ON p.product_code = a.product_code
        JOIN dim_employee em    ON em.employee_code = a.last_emp
        JOIN dim_loan_stage s   ON s.stage_name = a.last_stage
    ) s ON f.application_no = s.application_no
    WHEN MATCHED THEN UPDATE SET
        employee_key = s.employee_key, current_stage_key = s.stage_key,
        docs_verified_date_key = COALESCE(TO_CHAR(s.d_doc,'YYYYMMDD')::INT, 99991231),
        scored_date_key   = COALESCE(TO_CHAR(s.d_scr,'YYYYMMDD')::INT, 99991231),
        decision_date_key = COALESCE(TO_CHAR(s.d_dec,'YYYYMMDD')::INT, 99991231),
        signed_date_key   = COALESCE(TO_CHAR(s.d_sig,'YYYYMMDD')::INT, 99991231),
        disbursed_date_key= COALESCE(TO_CHAR(s.d_dis,'YYYYMMDD')::INT, 99991231),
        withdrawn_date_key= COALESCE(TO_CHAR(s.d_wdr,'YYYYMMDD')::INT, 99991231),
        approved_amount_uzs = s.apr_amt, disbursed_amount_uzs = s.dis_amt, credit_score = s.score,
        days_submit_to_docs = s.d_doc - s.d_sub, days_docs_to_scored = s.d_scr - s.d_doc,
        days_scored_to_decision = s.d_dec - s.d_scr, days_decision_to_signed = s.d_sig - s.d_dec,
        days_signed_to_disbursed = s.d_dis - s.d_sig,
        days_approval_to_disbursement = s.d_dis - s.d_dec,
        days_total_to_disbursement = s.d_dis - s.d_sub,
        last_event_ts = s.last_ts
    WHEN NOT MATCHED THEN INSERT (
        application_no, customer_key, branch_key, loan_product_key, employee_key, current_stage_key,
        submitted_date_key, docs_verified_date_key, scored_date_key, decision_date_key, signed_date_key,
        disbursed_date_key, withdrawn_date_key, requested_amount_uzs, approved_amount_uzs, disbursed_amount_uzs,
        credit_score, days_submit_to_docs, days_docs_to_scored, days_scored_to_decision,
        days_decision_to_signed, days_signed_to_disbursed, days_approval_to_disbursement,
        days_total_to_disbursement, last_event_ts)
    VALUES (
        s.application_no, s.customer_key, s.branch_key, s.loan_product_key, s.employee_key, s.stage_key,
        COALESCE(TO_CHAR(s.d_sub,'YYYYMMDD')::INT, 99991231), COALESCE(TO_CHAR(s.d_doc,'YYYYMMDD')::INT, 99991231),
        COALESCE(TO_CHAR(s.d_scr,'YYYYMMDD')::INT, 99991231), COALESCE(TO_CHAR(s.d_dec,'YYYYMMDD')::INT, 99991231),
        COALESCE(TO_CHAR(s.d_sig,'YYYYMMDD')::INT, 99991231), COALESCE(TO_CHAR(s.d_dis,'YYYYMMDD')::INT, 99991231),
        COALESCE(TO_CHAR(s.d_wdr,'YYYYMMDD')::INT, 99991231), s.req_amt, s.apr_amt, s.dis_amt, s.score,
        s.d_doc - s.d_sub, s.d_scr - s.d_doc, s.d_dec - s.d_scr, s.d_sig - s.d_dec, s.d_dis - s.d_sig,
        s.d_dis - s.d_dec, s.d_dis - s.d_sub, s.last_ts);
    GET DIAGNOSTICS v_rows = ROW_COUNT;

    UPDATE etl_watermark SET last_loaded_at = p_run_ts WHERE job = 'loan_snapshot';
    RETURN v_rows;
END $$;

-- ---------- Test data: 11 applications ----------
-- loaded_at = the night after the event (nightly file at 01:00)
INSERT INTO src_loan_event (application_no, stage, event_ts, employee_code, client_no, branch_code,
                            product_code, amount_uzs, credit_score, loaded_at)
SELECT v.app, v.stage, v.d::TIMESTAMP + TIME '10:00', v.emp,
       CASE WHEN v.stage = 'submitted' THEN v.cli END,
       CASE WHEN v.stage = 'submitted' THEN v.br  END,
       CASE WHEN v.stage = 'submitted' THEN v.pr  END,
       v.amt, CASE WHEN v.stage = 'scored' THEN 700 END,
       (v.d + 1)::TIMESTAMP + TIME '01:00'
FROM (VALUES
 ('L1','submitted','2026-07-01'::DATE,'E-01',88213,'B-001','CONS',20000000),
 ('L1','docs_verified','2026-07-02','E-01',NULL,NULL,NULL,NULL),
 ('L1','scored','2026-07-03','E-02',NULL,NULL,NULL,NULL),
 ('L1','approved','2026-07-04','E-02',NULL,NULL,NULL,18000000),
 ('L1','signed','2026-07-06','E-01',NULL,NULL,NULL,NULL),
 ('L1','disbursed','2026-07-08','E-03',NULL,NULL,NULL,18000000),
 ('L2','submitted','2026-07-10','E-01',93110,'B-001','CONS',30000000),
 ('L2','docs_verified','2026-07-12','E-01',NULL,NULL,NULL,NULL),
 ('L2','scored','2026-07-13','E-02',NULL,NULL,NULL,NULL),
 ('L2','approved','2026-07-15','E-02',NULL,NULL,NULL,30000000),
 ('L2','signed','2026-07-20','E-01',NULL,NULL,NULL,NULL),
 ('L2','disbursed','2026-07-25','E-03',NULL,NULL,NULL,30000000),
 ('L3','submitted','2026-08-01','E-01',88213,'B-001','MORT',400000000),
 ('L3','docs_verified','2026-08-10','E-01',NULL,NULL,NULL,NULL),
 ('L3','scored','2026-08-12','E-02',NULL,NULL,NULL,NULL),
 ('L3','approved','2026-08-20','E-02',NULL,NULL,NULL,380000000),
 ('L3','signed','2026-08-25','E-01',NULL,NULL,NULL,NULL),
 ('L3','disbursed','2026-09-05','E-03',NULL,NULL,NULL,380000000),
 ('L4','submitted','2026-07-05','E-01',91002,'B-010','CONS',10000000),
 ('L4','docs_verified','2026-07-06','E-01',NULL,NULL,NULL,NULL),
 ('L4','scored','2026-07-07','E-02',NULL,NULL,NULL,NULL),
 ('L4','approved','2026-07-08','E-02',NULL,NULL,NULL,10000000),
 ('L4','signed','2026-07-09','E-01',NULL,NULL,NULL,NULL),
 ('L4','disbursed','2026-07-10','E-03',NULL,NULL,NULL,10000000),
 ('L5','submitted','2026-08-03','E-01',91002,'B-010','AUTO',90000000),
 ('L5','docs_verified','2026-08-05','E-01',NULL,NULL,NULL,NULL),
 ('L5','scored','2026-08-06','E-02',NULL,NULL,NULL,NULL),
 ('L5','approved','2026-08-09','E-02',NULL,NULL,NULL,85000000),
 ('L5','signed','2026-08-12','E-01',NULL,NULL,NULL,NULL),
 ('L5','disbursed','2026-08-20','E-03',NULL,NULL,NULL,85000000),
 ('L6','submitted','2026-08-15','E-01',91002,'B-010','MORT',500000000),
 ('L6','docs_verified','2026-08-30','E-01',NULL,NULL,NULL,NULL),
```

```
 ('L6','scored','2026-09-02','E-02',NULL,NULL,NULL,NULL),
 ('L6','approved','2026-09-10','E-02',NULL,NULL,NULL,450000000),
 ('L6','signed','2026-09-15','E-01',NULL,NULL,NULL,NULL),
 ('L6','disbursed','2026-09-30','E-03',NULL,NULL,NULL,450000000),
 ('L7','submitted','2026-07-02','E-01',93110,'B-002','CONS',15000000),
 ('L7','docs_verified','2026-07-03','E-01',NULL,NULL,NULL,NULL),
 ('L7','scored','2026-07-04','E-02',NULL,NULL,NULL,NULL),
 ('L7','rejected','2026-07-05','E-02',NULL,NULL,NULL,NULL),
 ('L8','submitted','2026-08-07','E-01',93110,'B-002','AUTO',60000000),
 ('L8','docs_verified','2026-08-08','E-01',NULL,NULL,NULL,NULL),
 ('L8','withdrawn','2026-08-12','E-01',NULL,NULL,NULL,NULL),
 ('L9','submitted','2026-09-01','E-01',88213,'B-002','CONS',12000000),
 ('L10','submitted','2026-09-05','E-01',93110,'B-002','MORT',300000000),
 ('L10','docs_verified','2026-09-08','E-01',NULL,NULL,NULL,NULL),
 ('L10','scored','2026-09-10','E-02',NULL,NULL,NULL,NULL),
 ('L10','approved','2026-09-12','E-02',NULL,NULL,NULL,280000000),
 ('L11','submitted','2026-09-20','E-01',88213,'B-001','CONS',25000000),
 ('L11','docs_verified','2026-09-21','E-01',NULL,NULL,NULL,NULL),
 ('L11','scored','2026-09-22','E-02',NULL,NULL,NULL,NULL),
 ('L11','approved','2026-09-23','E-02',NULL,NULL,NULL,25000000),
 ('L11','signed','2026-09-28','E-01',NULL,NULL,NULL,NULL),
 ('L11','disbursed','2026-10-05','E-03',NULL,NULL,NULL,25000000)
) AS v(app, stage, d, emp, cli, br, pr, amt);
```

part3_queries.sql

```
SET search_path TO bank;

\echo '=== 3.2 night 1 (run at 2026-09-15 01:00): rows inserted/updated'
SELECT load_loan_snapshot('2026-09-15 01:00') AS rows_touched;
SELECT COUNT(*) AS rows_in_fact FROM fact_loan_application;

\echo '=== 3.2 night 2 (run at 2026-10-06 01:00): only applications with new events'
SELECT load_loan_snapshot('2026-10-06 01:00') AS rows_touched;
SELECT application_no FROM fact_loan_application WHERE last_event_ts >= '2026-09-15 01:00' ORDER BY 1;

\echo '=== 3.2 re-running the same night changes nothing (idempotent)'
SELECT load_loan_snapshot('2026-10-06 01:00') AS rows_touched;

\echo '=== 3.1 one row per application'
SELECT f.application_no, p.product_code, b.branch_code, s.stage_name AS current_stage,
       f.submitted_date_key, f.decision_date_key, f.disbursed_date_key, f.days_total_to_disbursement
FROM fact_loan_application f
JOIN dim_loan_product p ON p.loan_product_key = f.loan_product_key
JOIN dim_branch b       ON b.branch_key = f.branch_key
JOIN dim_loan_stage s   ON s.stage_key = f.current_stage_key
ORDER BY f.submitted_date_key;

\echo '=== Q4: where do applications stall? average days per step, by product'
SELECT p.product_code,
       ROUND(AVG(days_submit_to_docs),1)     AS submit_to_docs,
       ROUND(AVG(days_docs_to_scored),1)     AS docs_to_scored,
       ROUND(AVG(days_scored_to_decision),1) AS scored_to_decision,
       ROUND(AVG(days_decision_to_signed),1) AS decision_to_signed,
       ROUND(AVG(days_signed_to_disbursed),1) AS signed_to_disbursed
FROM fact_loan_application f JOIN dim_loan_product p USING (loan_product_key)
GROUP BY p.product_code ORDER BY p.product_code;

\echo '=== open applications and how long they have been waiting (age computed at query time, as of 2026-10-06)'
SELECT f.application_no, s.stage_name, DATE '2026-10-06' - TO_DATE(f.last_event_ts::DATE::TEXT,'YYYY-MM-DD') AS days_in_stage
FROM fact_loan_application f JOIN dim_loan_stage s ON s.stage_key = f.current_stage_key
WHERE NOT s.is_terminal ORDER BY days_in_stage DESC;

\echo '=== 3.3 stage counts at month end, rebuilt from the accumulating snapshot (clumsy)'
WITH month_ends AS (SELECT date_key AS me FROM dim_date WHERE year = 2026 AND month_num IN (7,8,9) AND is_month_end),
staged AS (
    SELECT m.me AS month_end,
           CASE WHEN f.submitted_date_key > m.me THEN NULL
                WHEN f.withdrawn_date_key  <= m.me THEN 'withdrawn'
                WHEN f.disbursed_date_key  <= m.me THEN 'disbursed'
                WHEN f.signed_date_key     <= m.me THEN 'signed'
                WHEN f.decision_date_key   <= m.me THEN (CASE WHEN s.stage_name = 'rejected' THEN 'rejected' ELSE 'approved' END)
                WHEN f.scored_date_key     <= m.me THEN 'scored'
                WHEN f.docs_verified_date_key <= m.me THEN 'docs_verified'
                ELSE 'submitted' END AS stage_at_month_end
    FROM fact_loan_application f
    JOIN dim_loan_stage s ON s.stage_key = f.current_stage_key
    CROSS JOIN month_ends m
)
SELECT month_end, stage_at_month_end, COUNT(*) AS applications
FROM staged WHERE stage_at_month_end IS NOT NULL
GROUP BY month_end, stage_at_month_end ORDER BY month_end, stage_at_month_end;

\echo '=== 3.4 median days approval -> disbursement by branch, disbursed in Q3 2026'
SELECT b.branch_name,
       COUNT(*) AS loans,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY f.days_approval_to_disbursement) AS median_days
FROM fact_loan_application f
JOIN dim_branch b ON b.branch_key = f.branch_key
JOIN dim_date d   ON d.date_key = f.disbursed_date_key
WHERE d.year = 2026 AND d.month_num BETWEEN 7 AND 9
GROUP BY b.branch_name ORDER BY b.branch_name;

\echo '=== 3.4 nightly data-quality test: milestones in order (must be 0 rows)'
```

```
SELECT application_no FROM fact_loan_application
WHERE (docs_verified_date_key < submitted_date_key)
   OR (scored_date_key < docs_verified_date_key AND scored_date_key <> 99991231)
   OR (decision_date_key < scored_date_key AND decision_date_key <> 99991231)
   OR (signed_date_key < decision_date_key AND signed_date_key <> 99991231)
   OR (disbursed_date_key < signed_date_key AND disbursed_date_key <> 99991231)
   OR (disbursed_date_key <> 99991231 AND signed_date_key = 99991231)   -- cannot disburse before signing
   OR days_approval_to_disbursement < 0;
```

part4_vault.sql

```
-- Part 4: Raw Data Vault for the customer (all identifiers below are invented)
DROP SCHEMA IF EXISTS vault CASCADE;
CREATE SCHEMA vault;
SET search_path TO vault;

-- ---------- Hubs ----------
-- Business key = PINFL (national personal ID). The only identifier that all
-- five systems can be mapped to, it never changes, it is unique per person.
CREATE TABLE hub_customer (
    customer_hk   CHAR(32) PRIMARY KEY,           -- md5(upper(trim(pinfl)))
    pinfl         CHAR(14) NOT NULL UNIQUE,       -- RESTRICTED zone only
    load_date     TIMESTAMP NOT NULL,
    record_source TEXT NOT NULL
);

-- Source identifiers: client_no / card_holder_id / app_user_id / crm_id
CREATE TABLE hub_source_customer (
    src_customer_hk    CHAR(32) PRIMARY KEY,      -- md5(system || '|' || id)
    source_system      TEXT NOT NULL,             -- ABS / CARD / APP / CRM
    source_customer_id TEXT NOT NULL,
    load_date          TIMESTAMP NOT NULL,
    record_source      TEXT NOT NULL,
    UNIQUE (source_system, source_customer_id)
);

CREATE TABLE hub_account (
    account_hk    CHAR(32) PRIMARY KEY,
    account_no    TEXT NOT NULL UNIQUE,
    load_date     TIMESTAMP NOT NULL,
    record_source TEXT NOT NULL
);

-- ---------- Links ----------
-- Same-as link: filled from the KYC system, which knows PINFL and every source ID
CREATE TABLE link_customer_identity (
    link_hk         CHAR(32) PRIMARY KEY,         -- md5(customer_hk || src_customer_hk)
    customer_hk     CHAR(32) NOT NULL REFERENCES hub_customer,
    src_customer_hk CHAR(32) NOT NULL REFERENCES hub_source_customer,
    load_date       TIMESTAMP NOT NULL,
    record_source   TEXT NOT NULL,
    UNIQUE (customer_hk, src_customer_hk)
);

CREATE TABLE link_customer_account (
    link_hk         CHAR(32) PRIMARY KEY,
    src_customer_hk CHAR(32) NOT NULL REFERENCES hub_source_customer,
    account_hk      CHAR(32) NOT NULL REFERENCES hub_account,
    load_date       TIMESTAMP NOT NULL,
    record_source   TEXT NOT NULL,
    UNIQUE (src_customer_hk, account_hk)
);

-- ---------- Satellites (insert-only; one row per CHANGE) ----------
CREATE TABLE sat_kyc_customer (                   -- on hub_customer, RESTRICTED
    customer_hk   CHAR(32) NOT NULL REFERENCES hub_customer,
    load_date     TIMESTAMP NOT NULL,
    hash_diff     CHAR(32) NOT NULL,
    record_source TEXT NOT NULL,
    full_name     TEXT, passport_no TEXT, risk_rating TEXT, sanctions_flag BOOLEAN,
    PRIMARY KEY (customer_hk, load_date)
);
CREATE TABLE sat_abs_customer (                   -- core banking
    src_customer_hk CHAR(32) NOT NULL REFERENCES hub_source_customer,
    load_date     TIMESTAMP NOT NULL,
    hash_diff     CHAR(32) NOT NULL,
    record_source TEXT NOT NULL,
    branch_code   TEXT, client_status TEXT,
    PRIMARY KEY (src_customer_hk, load_date)
);
CREATE TABLE sat_card_customer (                  -- card processing
    src_customer_hk CHAR(32) NOT NULL REFERENCES hub_source_customer,
    load_date     TIMESTAMP NOT NULL,
    hash_diff     CHAR(32) NOT NULL,
    record_source TEXT NOT NULL,
    source_changed_at TIMESTAMP,
    address       TEXT,
    PRIMARY KEY (src_customer_hk, load_date)
);
CREATE TABLE sat_app_customer (                   -- mobile app
    src_customer_hk CHAR(32) NOT NULL REFERENCES hub_source_customer,
    load_date     TIMESTAMP NOT NULL,
    hash_diff     CHAR(32) NOT NULL,
    record_source TEXT NOT NULL,
```

```
    phone         TEXT, device_id TEXT,
    PRIMARY KEY (src_customer_hk, load_date)
);
CREATE TABLE sat_crm_customer (                   -- CRM
    src_customer_hk CHAR(32) NOT NULL REFERENCES hub_source_customer,
    load_date     TIMESTAMP NOT NULL,             -- when WE loaded it
    hash_diff     CHAR(32) NOT NULL,
    record_source TEXT NOT NULL,
    source_changed_at TIMESTAMP,                  -- when the SOURCE changed it
    segment       TEXT, address TEXT, relationship_manager TEXT,
    PRIMARY KEY (src_customer_hk, load_date)
);

-- ---------- Sample load ----------
INSERT INTO hub_customer VALUES (md5('12345678901234'), '12345678901234', '2025-11-10 02:00', 'KYC');
INSERT INTO hub_source_customer
SELECT md5(s || '|' || i), s, i, '2025-11-10 02:00', s
FROM (VALUES ('ABS','88213'),('CARD','5519'),('APP','7c1e-demo'),('CRM','C-2290')) v(s,i);
INSERT INTO link_customer_identity
SELECT md5(md5('12345678901234') || h.src_customer_hk), md5('12345678901234'), h.src_customer_hk,
       '2025-11-10 02:00', 'KYC'
FROM hub_source_customer h;

INSERT INTO sat_kyc_customer VALUES
 (md5('12345678901234'), '2025-11-10 02:00', md5('Aziz T.|AA0000000|low|f'), 'KYC', 'Aziz T.', 'AA0000000', 'low', FALSE);
INSERT INTO sat_abs_customer VALUES
 (md5('ABS|88213'), '2025-11-10 02:00', md5('B-001|active'), 'ABS', 'B-001', 'active');
INSERT INTO sat_app_customer VALUES
 (md5('APP|7c1e-demo'), '2025-11-10 02:00', md5('+998901112233|dev-1'), 'APP', '+998901112233', 'dev-1');

-- Generic insert-only loader for CRM / CARD: add a row only if hash_diff changed
CREATE OR REPLACE FUNCTION load_crm(p_id TEXT, p_load TIMESTAMP, p_changed TIMESTAMP,
                                    p_segment TEXT, p_address TEXT, p_rm TEXT) RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE hk CHAR(32) := md5('CRM|' || p_id); hd CHAR(32) := md5(p_segment||'|'||p_address||'|'||p_rm); n INT;
BEGIN
    INSERT INTO sat_crm_customer (src_customer_hk, load_date, hash_diff, record_source, source_changed_at,
                                  segment, address, relationship_manager)
    SELECT hk, p_load, hd, 'CRM', p_changed, p_segment, p_address, p_rm
    WHERE hd IS DISTINCT FROM (SELECT hash_diff FROM sat_crm_customer WHERE src_customer_hk = hk
                               ORDER BY load_date DESC LIMIT 1);
    GET DIAGNOSTICS n = ROW_COUNT; RETURN n;
END $$;

CREATE OR REPLACE FUNCTION load_card(p_id TEXT, p_load TIMESTAMP, p_changed TIMESTAMP, p_address TEXT) RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE hk CHAR(32) := md5('CARD|' || p_id); hd CHAR(32) := md5(p_address); n INT;
BEGIN
    INSERT INTO sat_card_customer (src_customer_hk, load_date, hash_diff, record_source, source_changed_at, address)
    SELECT hk, p_load, hd, 'CARD', p_changed, p_address
    WHERE hd IS DISTINCT FROM (SELECT hash_diff FROM sat_card_customer WHERE src_customer_hk = hk
                               ORDER BY load_date DESC LIMIT 1);
    GET DIAGNOSTICS n = ROW_COUNT; RETURN n;
END $$;

SELECT load_crm ('C-2290','2025-11-11 02:00','2025-11-10 09:00','mass',     'Chilonzor 5, Tashkent', 'RM-7');
SELECT load_card('5519',   '2025-11-10 02:00','2025-11-10 09:00',            'Chilonzor 5, Tashkent');
-- 2026-02-08 the CRM segment changes, but the file arrives late (loaded 02-12)
SELECT load_crm ('C-2290','2026-02-12 02:00','2026-02-08 14:00','affluent', 'Chilonzor 5, Tashkent', 'RM-7');
-- 2026-05-03 the card system receives a new address; the CRM still sends the old one
SELECT load_card('5519',   '2026-05-03 02:00','2026-05-03 11:00',            'Yunusobod 12, Tashkent');
SELECT load_crm ('C-2290','2026-05-04 02:00','2026-02-08 14:00','affluent', 'Chilonzor 5, Tashkent', 'RM-7');  -- no change -> 0 rows
```

part4_queries.sql

```
SET search_path TO vault;

\echo '=== 4.2 sat_card_customer: two rows, nothing overwritten'
SELECT load_date, LEFT(hash_diff,8) AS hash_diff, record_source, address FROM sat_card_customer ORDER BY load_date;
\echo '=== 4.2 sat_crm_customer: the CRM still holds the old address (no new row on 05-04)'
SELECT load_date, LEFT(hash_diff,8) AS hash_diff, record_source, segment, address FROM sat_crm_customer ORDER BY load_date;

-- ---------- 4.3 address rule ----------
-- Most recent change wins (source_changed_at, else load_date). Ties: CRM > CARD.
CREATE OR REPLACE VIEW v_customer_address AS
WITH candidates AS (
    SELECT l.customer_hk, s.address, 'CARD' AS address_source,
           COALESCE(s.source_changed_at, s.load_date) AS changed_at, 2 AS precedence, s.load_date
    FROM sat_card_customer s JOIN link_customer_identity l ON l.src_customer_hk = s.src_customer_hk
    UNION ALL
    SELECT l.customer_hk, s.address, 'CRM',
           COALESCE(s.source_changed_at, s.load_date), 1, s.load_date
    FROM sat_crm_customer s JOIN link_customer_identity l ON l.src_customer_hk = s.src_customer_hk
), ranked AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_hk ORDER BY changed_at DESC, precedence) AS rn
    FROM candidates WHERE address IS NOT NULL
)
SELECT customer_hk, address, address_source, changed_at AS address_changed_at
FROM ranked WHERE rn = 1;

\echo '=== 4.3 resolved address with the winning source'
SELECT address, address_source, address_changed_at FROM v_customer_address;

-- ---------- 4.4 auditor ----------
```

```
\echo '=== 4.4 what did the CRM say the segment was on 2026-02-10, and when did we load it?'
SELECT s.segment, s.source_changed_at, s.load_date
FROM sat_crm_customer s
JOIN hub_source_customer h ON h.src_customer_hk = s.src_customer_hk
WHERE h.source_system = 'CRM' AND h.source_customer_id = 'C-2290'
  AND s.source_changed_at <= TIMESTAMP '2026-02-10 23:59:59'
ORDER BY s.source_changed_at DESC LIMIT 1;

\echo '=== 4.4 what did WE know on 2026-02-10 (what had been loaded by then)?'
SELECT s.segment, s.load_date
FROM sat_crm_customer s
JOIN hub_source_customer h ON h.src_customer_hk = s.src_customer_hk
WHERE h.source_system = 'CRM' AND h.source_customer_id = 'C-2290'
  AND s.load_date <= TIMESTAMP '2026-02-10 23:59:59'
ORDER BY s.load_date DESC LIMIT 1;

\echo '=== vault integrity: every source id maps to exactly one customer (must be 0 rows)'
SELECT src_customer_hk FROM link_customer_identity GROUP BY src_customer_hk HAVING COUNT(*) > 1;
```

part5_pipeline.sql

```
-- Part 5: incremental load, quality tests, cost
SET search_path TO bank;
DROP TABLE IF EXISTS stg_core_txn, load_log, fact_transaction_audit, agg_daily_inflow, src_control CASCADE;

-- Immutable landing area: every row of every file is kept as received
CREATE TABLE stg_core_txn (
    row_seq        BIGSERIAL PRIMARY KEY,          -- order of arrival
    file_id        TEXT NOT NULL,
    transaction_id TEXT NOT NULL,
    leg_no         SMALLINT NOT NULL,
    date_key       INTEGER NOT NULL,
    account_key    INTEGER NOT NULL,
    customer_key   INTEGER NOT NULL,
    branch_key     INTEGER NOT NULL,
    txn_type_key   INTEGER NOT NULL,
    channel_key    INTEGER NOT NULL,
    amount_uzs     NUMERIC(18,2) NOT NULL
);
CREATE TABLE src_control (file_id TEXT PRIMARY KEY, row_count INTEGER NOT NULL, total_amount NUMERIC(18,2) NOT NULL);
CREATE TABLE load_log (
    file_id TEXT PRIMARY KEY, loaded_at TIMESTAMP NOT NULL,
    rows_inserted INT, rows_corrected INT, rows_unchanged INT, rows_duplicate INT
);
-- Old values of corrected rows (audit trail)
CREATE TABLE fact_transaction_audit (
    txn_leg_id BIGINT, old_amount_uzs NUMERIC(18,2), new_amount_uzs NUMERIC(18,2),
    changed_by_file TEXT, changed_at TIMESTAMP DEFAULT now()
);
-- Pre-aggregate for Question 1 (net inflow by day, region, segment)
CREATE TABLE agg_daily_inflow (
    date_key INTEGER, region TEXT, segment TEXT, net_inflow_uzs NUMERIC(18,2),
    PRIMARY KEY (date_key, region, segment)
);

CREATE OR REPLACE FUNCTION rebuild_agg(p_dates INTEGER[]) RETURNS VOID LANGUAGE sql AS $$
    DELETE FROM agg_daily_inflow WHERE date_key = ANY(p_dates);
    INSERT INTO agg_daily_inflow
    SELECT f.date_key, b.region, c.segment, SUM(f.amount_uzs)
    FROM fact_transaction f
    JOIN dim_branch b   ON b.branch_key = f.branch_key
    JOIN dim_customer c ON c.customer_key = f.customer_key
    WHERE f.date_key = ANY(p_dates)
    GROUP BY f.date_key, b.region, c.segment;
$$;

CREATE OR REPLACE FUNCTION load_core_file(p_file TEXT, p_now TIMESTAMP) RETURNS TEXT
LANGUAGE plpgsql AS $$
DECLARE n_dup INT; n_ins INT; n_upd INT; n_same INT; v_dates INTEGER[];
BEGIN
    -- (a) never load a file twice
    IF EXISTS (SELECT 1 FROM load_log WHERE file_id = p_file) THEN RETURN 'skipped: file already loaded'; END IF;

    -- latest version of each (transaction_id, leg_no) inside the file
    CREATE TEMP TABLE batch ON COMMIT DROP AS
    SELECT DISTINCT ON (transaction_id, leg_no) *
    FROM stg_core_txn WHERE file_id = p_file
    ORDER BY transaction_id, leg_no, row_seq DESC;
    SELECT (SELECT COUNT(*) FROM stg_core_txn WHERE file_id = p_file) - COUNT(*) INTO n_dup FROM batch;

    -- days that really change: new rows or corrected amounts (computed BEFORE we modify the fact)
    SELECT ARRAY_AGG(DISTINCT s.date_key) INTO v_dates FROM batch s
     WHERE NOT EXISTS (SELECT 1 FROM fact_transaction f
                       WHERE f.transaction_id = s.transaction_id AND f.leg_no = s.leg_no
                         AND f.amount_uzs = s.amount_uzs);

    -- (b) corrections: keep the old value in the audit table, then update in place
    INSERT INTO fact_transaction_audit (txn_leg_id, old_amount_uzs, new_amount_uzs, changed_by_file)
    SELECT f.txn_leg_id, f.amount_uzs, s.amount_uzs, p_file
    FROM batch s JOIN fact_transaction f USING (transaction_id, leg_no)
    WHERE f.amount_uzs <> s.amount_uzs;
    GET DIAGNOSTICS n_upd = ROW_COUNT;

    UPDATE fact_transaction f SET amount_uzs = s.amount_uzs
```

```
    FROM batch s WHERE f.transaction_id = s.transaction_id AND f.leg_no = s.leg_no
      AND f.amount_uzs <> s.amount_uzs;

    SELECT COUNT(*) INTO n_same FROM batch s JOIN fact_transaction f USING (transaction_id, leg_no)
     WHERE f.amount_uzs = s.amount_uzs;

    INSERT INTO fact_transaction (transaction_id, leg_no, date_key, txn_ts, account_key, customer_key,
                                  branch_key, txn_type_key, channel_key, amount_uzs)
    SELECT s.transaction_id, s.leg_no, s.date_key, TO_DATE(s.date_key::TEXT,'YYYYMMDD'), s.account_key,
           s.customer_key, s.branch_key, s.txn_type_key, s.channel_key, s.amount_uzs
    FROM batch s
    WHERE NOT EXISTS (SELECT 1 FROM fact_transaction f WHERE f.transaction_id = s.transaction_id AND f.leg_no = s.leg_no);
    GET DIAGNOSTICS n_ins = ROW_COUNT;
    n_same := n_same - n_upd;

    -- (c) rebuild the aggregate ONLY for the days that changed (new rows or corrections)
    IF v_dates IS NOT NULL THEN PERFORM rebuild_agg(v_dates); END IF;

    INSERT INTO load_log VALUES (p_file, p_now, n_ins, n_upd, n_same, n_dup);
    RETURN format('inserted=%s corrected=%s unchanged=%s duplicates_in_file=%s', n_ins, n_upd, n_same, n_dup);
END $$;

-- ---------- Test files ----------
-- File F1 (2026-11-02 01:00): 3 transactions of 2026-11-01
INSERT INTO stg_core_txn (file_id, transaction_id, leg_no, date_key, account_key, customer_key, branch_key, txn_type_key, channel_key,
amount_uzs)
VALUES ('F1','N-1',1,20261101,1,1,1,1,1, 500), ('F1','N-2',1,20261101,2,2,3,1,1, 300), ('F1','N-3',1,20261101,3,3,2,2,2,-120);
INSERT INTO src_control VALUES ('F1', 3, 680);

-- File F2 (2026-11-03 01:00): new day + a CORRECTION of N-1 (500 -> 450) + a duplicate line + an unchanged re-send
INSERT INTO stg_core_txn (file_id, transaction_id, leg_no, date_key, account_key, customer_key, branch_key, txn_type_key, channel_key,
amount_uzs)
VALUES ('F2','N-4',1,20261102,1,1,1,1,1, 200),
       ('F2','N-4',1,20261102,1,1,1,1,1, 200),          -- duplicate line in the same file
       ('F2','N-5',1,20261102,2,2,3,2,2, -50),
       ('F2','N-1',1,20261101,1,1,1,1,1, 450),          -- correction of an earlier day
       ('F2','N-2',1,20261101,2,2,3,1,1, 300);          -- re-sent, unchanged
INSERT INTO src_control VALUES ('F2', 4, 900);          -- distinct rows 200 -50 450 300 = 900
```

part5_queries.sql

```
SET search_path TO bank;

\echo '=== 5.1 load F1, then F2, then F2 again'
SELECT load_core_file('F1', '2026-11-02 02:10');
SELECT load_core_file('F2', '2026-11-03 02:15');
SELECT load_core_file('F2', '2026-11-03 03:00');

\echo '=== 5.1 correction applied, old value kept in the audit table'
SELECT f.transaction_id, f.amount_uzs AS current_amount, a.old_amount_uzs, a.changed_by_file
FROM fact_transaction f JOIN fact_transaction_audit a USING (txn_leg_id);

\echo '=== 5.1 no duplicates in the fact (must be 0 rows)'
SELECT transaction_id, leg_no, COUNT(*) FROM fact_transaction GROUP BY 1,2 HAVING COUNT(*) > 1;

\echo '=== 5.1 yesterday''s aggregate for Question 1 (2026-11-01 reflects the correction: 450, not 500)'
SELECT date_key, region, segment, net_inflow_uzs FROM agg_daily_inflow WHERE date_key >= 20261101 ORDER BY 1,2,3;

-- ====================== 5.3 data quality tests ======================
\echo '=== T1 grain: one row per (transaction_id, leg_no) - must be 0 rows'
SELECT transaction_id, leg_no, COUNT(*) FROM fact_transaction GROUP BY 1,2 HAVING COUNT(*) > 1;

\echo '=== T2 referential integrity: facts whose customer key is missing in the dimension - must be 0 rows'
SELECT f.txn_leg_id FROM fact_transaction f LEFT JOIN dim_customer c ON c.customer_key = f.customer_key
WHERE c.customer_key IS NULL;

\echo '=== T3 no overlapping Type 2 rows - must be 0 rows'
SELECT a.client_no FROM dim_customer a JOIN dim_customer b
  ON a.client_no = b.client_no AND a.customer_key <> b.customer_key
 AND a.valid_from < b.valid_to AND b.valid_from < a.valid_to;

\echo '=== T4 reconciliation: rows and total of each loaded file vs the control file - must be 0 rows'
SELECT c.file_id, c.row_count, d.rows_loaded, c.total_amount, d.amount_loaded
FROM src_control c
JOIN (SELECT file_id, COUNT(*) AS rows_loaded, SUM(amount_uzs) AS amount_loaded
        FROM (SELECT DISTINCT ON (file_id, transaction_id, leg_no) * FROM stg_core_txn
                ORDER BY file_id, transaction_id, leg_no, row_seq DESC) x GROUP BY file_id) d USING (file_id)
WHERE c.row_count <> d.rows_loaded OR c.total_amount <> d.amount_loaded;

\echo '=== T5 freshness: last successful load older than 26 hours at 08:00 - must be 0 rows'
SELECT MAX(loaded_at) AS last_load, TIMESTAMP '2026-11-03 08:00' - MAX(loaded_at) AS age
FROM load_log HAVING TIMESTAMP '2026-11-03 08:00' - MAX(loaded_at) > INTERVAL '26 hours';

\echo '=== T5 same test, 2 days later (stale -> must fail, returns 1 row)'
SELECT MAX(loaded_at) AS last_load, TIMESTAMP '2026-11-05 08:00' - MAX(loaded_at) AS age
FROM load_log HAVING TIMESTAMP '2026-11-05 08:00' - MAX(loaded_at) > INTERVAL '26 hours';

-- ====================== 5.5 volume ======================
\echo '=== 5.5 volume'
SELECT 9000000::NUMERIC * 200 / 1e9 AS gb_per_day_raw,
       9000000::NUMERIC * 200 * 365 / 1e9 AS gb_per_year_raw,
       9000000::NUMERIC * 200 * 365 / 5 / 1e9 AS gb_per_year_parquet;
```